

Homework of Lesson 3: HW 2/1 – HW 2/4

Remark. The results used are stated in boxes before the exercise where they are first needed and are cited by number together with the displayed equation; each one is a result of Lesson 3 (2026 notebook; the cell is given in the box), or it is proved right after its box. The code of HW 2/1 is in `plotter.py` and `hw21_examples.py`; the code of HW 2/2–2/4 is in the notebooks `1_HW_EXE_2`, `1_HW_EXE_3` and `1_HW_EXE_4`, in the same order as here. In the new versions, each change is marked `# change # k ... # end change # k`, and a replaced line of the professor's code is kept as a `#Delete.` comment. All code shown can be copied from this PDF and pasted into Python (spaces and indentation are kept; there are no line numbers).

HW 2/1. Smart up our plotter to be able to customize the hard-coded layout setup inside the function. Add more custom feature that you would like to use.

Solution. We need to do:

- (1) Every layout setting that is written inside the body of the plotter of cell 9 must become a choice of the caller;
- (2) New options of our own.

What the plotter of cell 9 does.

The professor's code. Cell 9 of the 2026 notebook, verbatim (one long comment is split over two lines):

`professor_cell109.py`, lines 4–50

```
import matplotlib.pyplot as plt
from matplotlib import rcParams # detailed parameter setting
""" more details on customization: https://matplotlib.org/stable/tutorials/introductory/customizing.html """
from typing import Dict, List, Union

# some type hints in the header of the function
def my_plotter(x: List[float], y: Union[List[float], List[List[float]]], layout: Dict = {}, names: List[str] = None) -> None:
    """ inline for loop is called 'list comprehension' """
    y = [y] if all(isinstance(item, float) for item in y) else y

    plt.figure(figsize=(8, 4))
    lines = []
    show_legend = True if names is not None else False
    # show_legend = True if names else False -> here is fine, but be cautious bc 0, empty str and empty list
    # will be evaluated to False

    """ setup some basic key-word arguments for plot line """
    plot_kwargs = {
        'linestyle': 'solid',
        'linewidth': 4
    }
    if names is not None:
        show_legend = True
        if len(names) != len(y):
            raise ValueError("Length of names is not matching with number of plotted y lists.")

    """ 'enumerate' add a counter to the loop """
    for i, y_item in enumerate(y):
        if show_legend:
            plot_kwargs['label'] = names[i]
            _line = plt.plot(x, y_item, **plot_kwargs)
            lines.append(_line)

    if show_legend:
        plt.legend(fontsize=16)
    if 'title' in layout:
        plt.title(layout['title'], fontsize=20)
        rcParams['axes.titlepad'] = 30 # moving the title a little further away from the plot
    if 'x_label' in layout:
        plt.xlabel(layout['x_label'], fontsize=16)
        rcParams['axes.labelpad'] = 20 # moving the ax label a little further away from the plot

    """ enhance axes """
    ax = plt.gca() # gca: get current axes
    ax.axhline(linestyle='--', color='black', linewidth=1)
    plt.show()
```

Cells 10 and 11 of the notebook use it to plot the payoffs of a long and a short forward with $K = 100$ (the call of cell 11 is split over several lines):

hw21_examples.py, lines 12–27 (block 1)

```
import numpy as np

strike = 100
step_size = 0.001

x_stock_price = np.arange(0, 2 * strike, step_size)
y_forward_payoff = np.array([x - strike for x in x_stock_price])

y_forward_payoff__short = y_forward_payoff * (-1)
y_forward_payoff__short

layout = {'title': 'Forward', 'x_label': '$S_{T}$'}
my_plotter(x_stock_price, [y_forward_payoff, y_forward_payoff__short],
           layout=layout,
           names=['Long Forward Payoff at time $T$',
                  'Short Forward Payoff at time $T$'])
```

What it shows. The last line of cell 10 shows the array of short-forward payoffs (with NumPy 2.2, the version of the course environment; older versions omit the second line), and cell 11 shows Figure 1:

output of cell 10

```
array([100.    ,  99.999,  99.998, ..., -99.997, -99.998, -99.999],
      shape=(200000,))
```

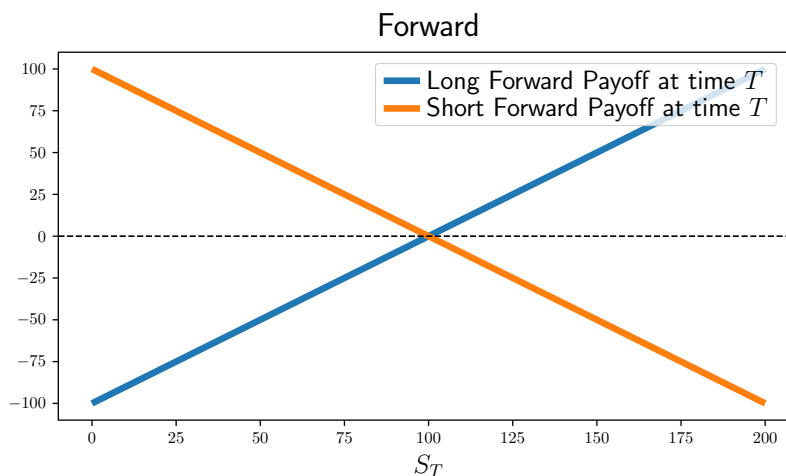


Figure 1: Output of cell 11 (block 1) in a new session: the plot drawn by the plotter of cell 9, as the notebook displays it. It is the same picture as the output stored in the course notebook.

Step by step. Figure 2 follows the code of cell 9 step by step. The orange boxes contain the hard-coded layout (Table 1); of the dictionary `layout` only the keys "title" and "x_label" are read.

Table 1: Hard-coded layout of cell 9 (orange tags of Figure 2).

tag	hard-coded value in cell 9	what the caller cannot change
H1	<code>figsize=(8, 4)</code>	size of the figure
H2	<code>linestyle='solid', linewidth=4</code>	style and width of the curves (also no colour, marker or opacity)
H3	<code>plt.legend(fontsize=16)</code>	legend font size (and its position)
H4	<code>fontsize=20, axes.titlepad = 30</code>	title size and its distance to the axes
H5	<code>fontsize=16, axes.labelpad = 20</code>	x -label size and distance (there is no y -label)
H6	dashed black line at $y = 0$, width 1	whether this line is drawn, and where

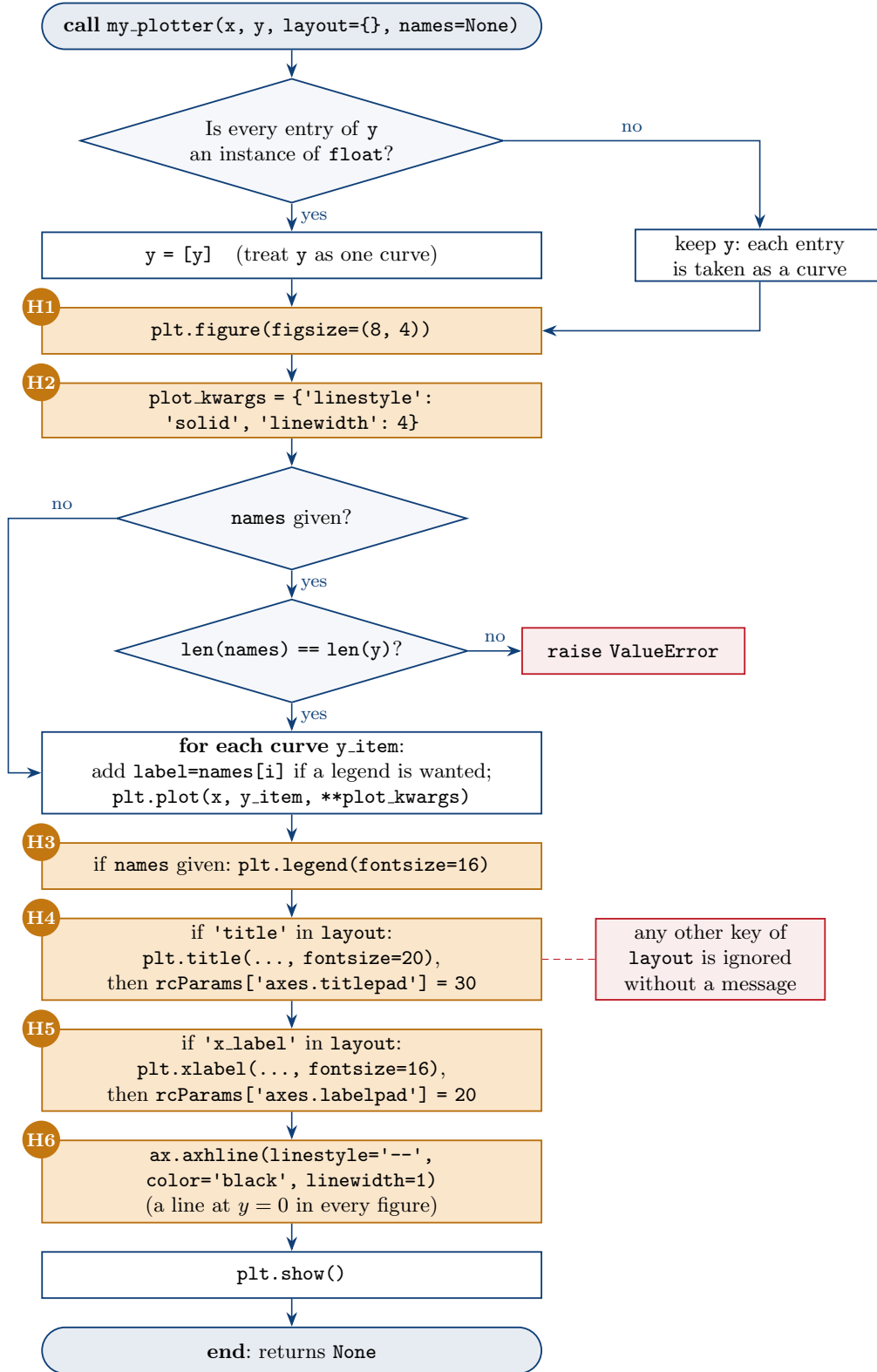


Figure 2: The plotter of cell 9. Orange: hard-coded layout (H1–H6).

Design.

We keep the interface of cell 9 and make the following choices.

- *One dictionary of defaults* (~~H1~~–~~H6~~). Every layout setting is one of the 25 keys of the dictionary `DEFAULT_LAYOUT`, and the default of each setting of cell 9 is its value in cell 9. Hence the calls of the notebook run unchanged and, without layout options, draw the figure of cell 9: same size, curves, fonts and line at $y = 0$. There are two differences: the paddings now act on the figure itself, as cell 9 intended, and `fig.tight_layout()` shrinks the axes so that the title and the x -label fit inside the 8×4 figure. For every key k the option used is

$$\text{opts}[k] = \begin{cases} \text{layout}[k], & \text{if the caller gives the key } k, \\ \text{DEFAULT_LAYOUT}[k], & \text{otherwise.} \end{cases}$$

The dictionary is in `plotter.py`; the comment [cell 9] marks a setting of cell 9, with the value of cell 9, and (new) a new option (`title` and `x_label` are the two keys read by cell 9, and `show` decides whether `plt.show()` is called):

`plotter.py`, lines 27–53

```
DEFAULT_LAYOUT: Dict[str, Any] = {
    "figsize": (8, 4),          # inches (width, height)           [cell 9]
    "title": None,             # title of the axes
    "x_label": None,           # x-axis label
    "y_label": None,           # y-axis label              (new)
    "title_fontsize": 20,      #                               [cell 9]
    "label_fontsize": 16,      # x- and y-label            [cell 9]
    "legend_fontsize": 16,     #                               [cell 9]
    "tick_fontsize": None,     # None: Matplotlib default  [cell 9]
    "title_pad": 30,           # points between title and axes [cell 9]
    "label_pad": 20,           # points between label and axis [cell 9]
    "linewidth": 4,            #                               [cell 9]
    "colors": None,            # colour or list, cycled over curves (new)
    "linestyles": "solid",     # style or list like ["-", "--"] [cell 9]
    "markers": None,           # marker or list like [None, "o"] (new)
    "alpha": 1.0,              # opacity of the curves        (new)
    "legend_loc": "best",      # position of the legend        (new)
    "grid": False,             # True: light dashed grid       (new)
    "hline": 0.0,              # y of the dashed black line, or None [cell 9]
    "vlines": None,            # x values of dotted vertical lines (new)
    "xlim": None,              # (xmin, xmax)                  (new)
    "ylim": None,              # (ymin, ymax)                  (new)
    "logx": False,             # logarithmic x-axis             (new)
    "logy": False,             # logarithmic y-axis             (new)
    "save_path": None,         # e.g. "figures/plot.pdf"       (new)
    "show": None,              # None: show only a figure created here
}
```

The merge rule above, with the `KeyError` for unknown keys, is the start of `my_plotter`:

`plotter.py`, lines 77–82

```
opts = dict(DEFAULT_LAYOUT)
for key, value in (layout or {}).items():
    if key not in DEFAULT_LAYOUT:
        raise KeyError(f"unknown layout key {key!r}; "
                       f"allowed keys: {sorted(DEFAULT_LAYOUT)}")
    opts[key] = value
```

Figure 3 follows the new function; the green tags show where each hard-coded value of Figure 2 went.

The same call with the new plotter.

After `from plotter import my_plotter`, the call of cell 11 runs unchanged; we only keep the returned pair:

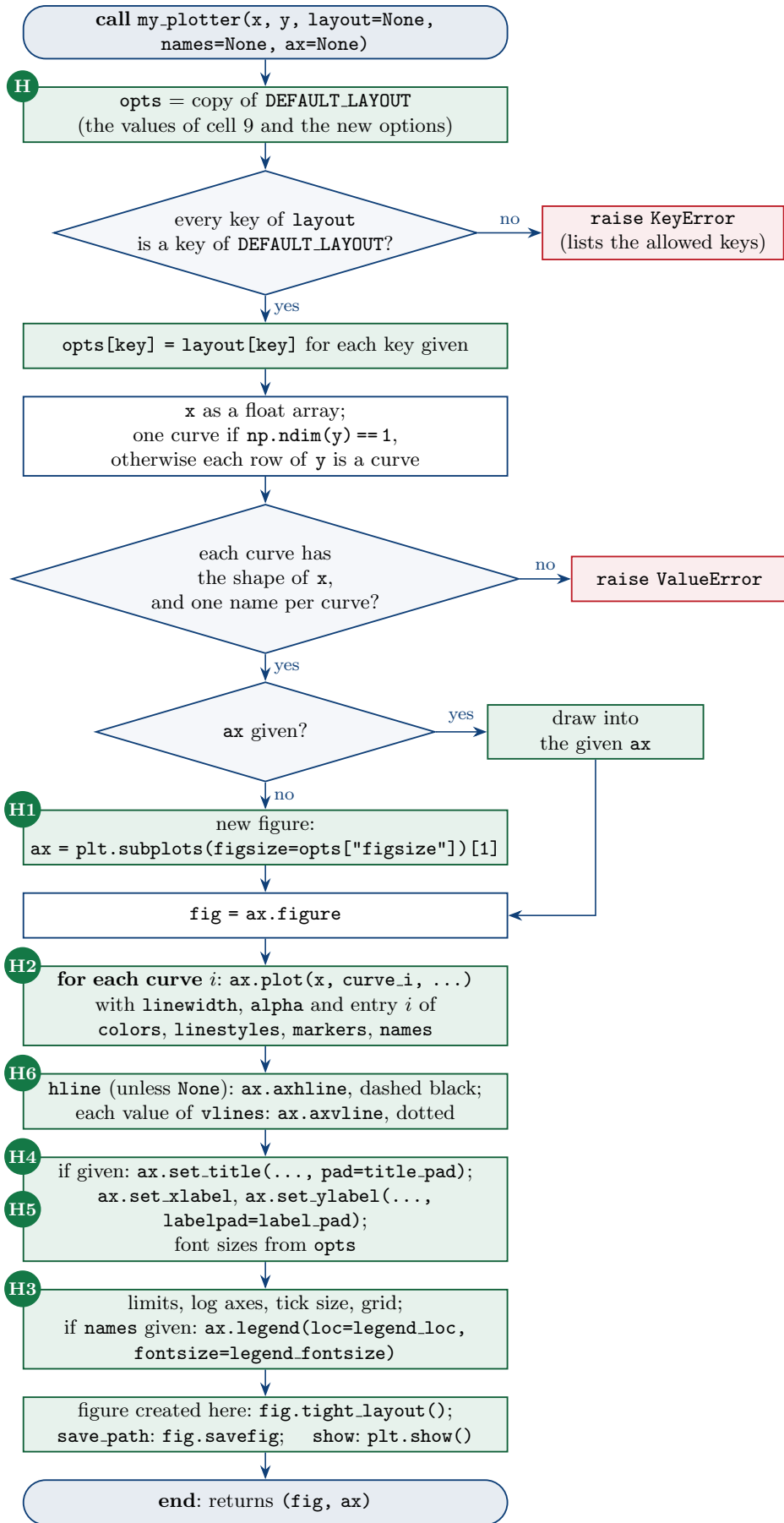


Figure 3: The new `my_plotter` of `plotter.py`. Green boxes use the options `opts`; green tags: where the hard-coded values **H1**–**H6** of Figure 2 went (**H**: all of them, as defaults).

hw21.examples.py, lines 30–35 (block 2)

```
from plotter import my_plotter          # the new plotter replaces cell 9

fig, ax = my_plotter(x_stock_price, [y_forward_payoff, y_forward_payoff__short],
                    layout=layout,
                    names=['Long Forward Payoff at time  $T$ ',
                          'Short Forward Payoff at time  $T$ '])
```

What it shows is Figure 4. Compared with Figure 1, the figure size, the curves, the colours, the font sizes and the line at $y = 0$ are the same, because these are the defaults. The title and the x -label are further from the axes: the new plotter applies the paddings 30 and 20 of cell 9 to this figure, while cell 9, in its first call, still draws with the default paddings (from its second call on, cell 9 also uses 30 and 20).

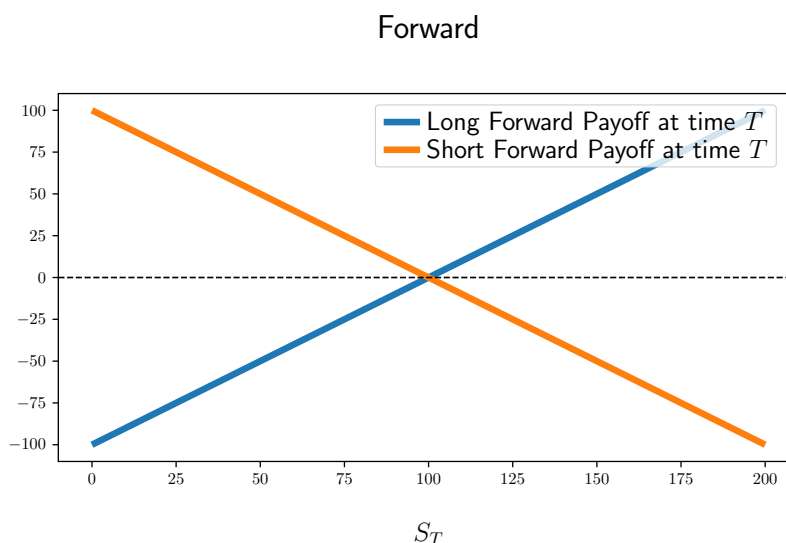


Figure 4: Output of block 2: the call of cell 11 with the new plotter and no layout options except the title and the x -label of cell 11.

How to use the new features.

Each feature is a key of `DEFAULT_LAYOUT`. The caller gives only the keys he wants to change, and the function passes each value to one Matplotlib command (Table 2). For `colors`, `linestyles` and `markers` a list is cycled over the curves: curve i gets the entry $i \bmod (\text{length of the list})$, so `["-", "--"]` alternates solid and dashed curves; a single string, such as `"red"`, applies to all curves.

For example, two plots side by side in one figure, saved to a file (a figure made by the caller keeps its layout, so the caller applies `fig.tight_layout()`); it shows Figure 5:

hw21.examples.py, lines 38–49 (block 3)

```
x = np.arange(0, 200, 0.001)
call_payoff = np.maximum(x - 100, 0.0)
put_payoff = np.maximum(100 - x, 0.0)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
my_plotter(x, [call_payoff, put_payoff], ax=axes[0], names=["call", "put"],
          layout={"title": "Payoffs", "colors": ["tab:blue", "tab:red"]})
my_plotter(x, put_payoff - 8, ax=axes[1], layout={"title": "Put net profit",
          "hline": None, "vlines": [92], "grid": True})
fig.tight_layout()
fig.savefig("figures/hw21_subplots.pdf")
plt.show()
```

Example.

Table 2: The new features of `my_plotter`: the key, an example value and the Matplotlib command that uses it.

group	key	example value	passed to
labels	<code>y_label</code>	"value at T "	<code>ax.set_ylabel</code>
	<code>tick_fontsize</code>	10	<code>ax.tick_params(labelsize=...)</code>
curves	<code>colors</code>	["tab:blue", "tab:red"]	<code>color=</code> of <code>ax.plot</code>
	<code>linestyles</code>	["-", "--"]	<code>linestyle=</code> of <code>ax.plot</code>
	<code>markers</code>	[None, "o"]	<code>marker=</code> of <code>ax.plot</code>
	<code>alpha</code>	0.6	<code>alpha=</code> of <code>ax.plot</code>
axes	<code>legend_loc</code>	"upper center"	<code>ax.legend(loc=...)</code>
	<code>grid</code>	True	<code>ax.grid</code>
	<code>hline</code>	None (no line) or 5.0	<code>ax.axhline</code>
	<code>vlines</code>	[92, 105]	<code>ax.axvline</code> , once per value
	<code>xlim, ylim</code>	(-15, 100)	<code>ax.set_xlim</code> , <code>ax.set_ylim</code>
	<code>logx, logy</code>	True	<code>ax.set_xscale("log")</code> , <code>ax.set_yscale("log")</code>
output	<code>save_path</code>	"payoffs.pdf"	<code>fig.savefig</code>
	<code>show</code>	False	True: always <code>plt.show()</code> ; default None: only for a figure created here
	<code>argument ax</code>	<code>axes[0]</code>	draws into that subplot
	<code>return value</code>	<code>fig, ax = my_plotter(...)</code>	the figure stays editable

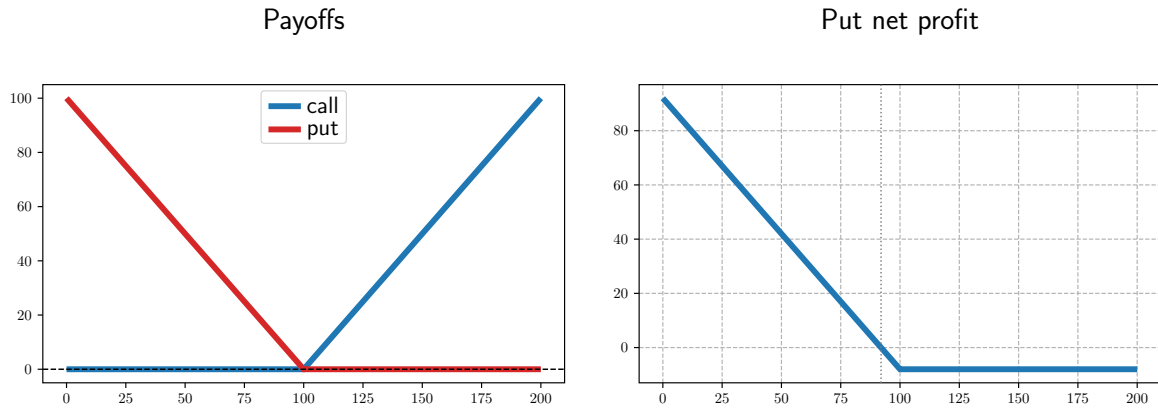


Figure 5: Output of block 3: two calls of the new plotter drawing into the two axes of one figure.

Figure 6 is drawn by one call of the new plotter with the data of cells 10, 19 and 25 ($K = 100$, call premium 5, put premium 8; cell 25 subtracts `call_price` from the put payoff by mistake; here the put premium is used). Its options are grouped as in Table 2; the code continues block 3 (it uses `x`, `call_payoff`, `put_payoff`) and shows Figure 6:

hw21.examples.py, lines 52–71 (block 4)

```
call_price, put_price = 5.0, 8.0          # premiums of cells 19 and 25
example_layout = {
    # text: title, axis labels, sizes and paddings
    "title": "Long call and long put, K = 100", "title_fontsize": 13,
    "x_label": "$S_T$", "y_label": "value at $T$", "label_fontsize": 12,
    "title_pad": 12, "label_pad": 8, "tick_fontsize": 10,
    # curves: one entry per curve (a list is cycled over the curves)
    "colors": ["tab:blue", "tab:blue", "tab:red", "tab:red"],
    "linestyles": ["-", "--", "-", "--"], "linewidth": 2.2,
    # axes: reference lines, limits, grid and legend
    "hline": None, "vlines": [92, 105], "ylim": (-15, 100), "grid": True,
    "legend_loc": "upper center", "legend_fontsize": 9,
    # output: figure size and file
    "figsize": (7.5, 3.8), "save_path": "figures/hw21_payoffs.pdf",
}
fig, ax = my_plotter(x, [call_payoff, call_payoff - call_price,
                          put_payoff, put_payoff - put_price],
                     layout=example_layout,
                     names=["call payoff", "call net profit (c = 5)",
                           "put payoff", "put net profit (p = 8)"])
```

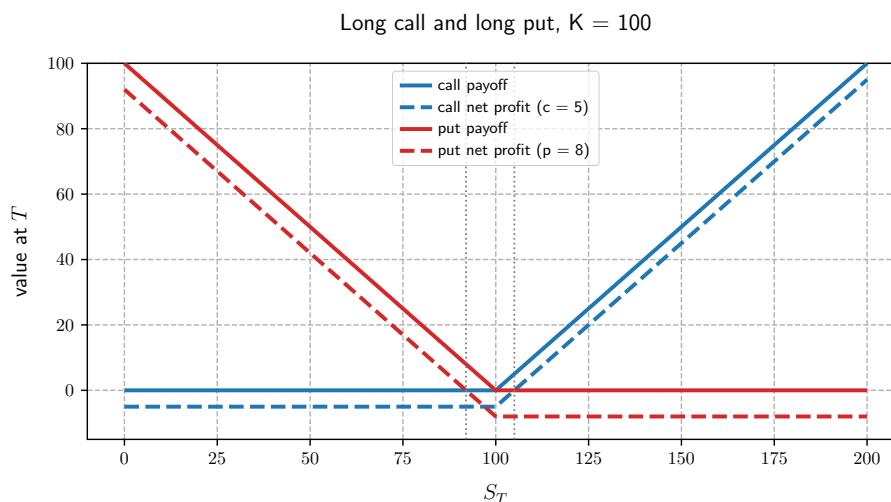


Figure 6: Output of block 4. Payoffs (solid) and net profits (dashed; payoff minus premium, cells 19–20 and 25–26) of a long call (blue, premium 5) and a long put (red, premium 8) with $K = 100$, drawn with the new plotter. The dotted lines (option `vlines`) are at the break-even points $S_T = 100 + 5 = 105$ and $S_T = 100 - 8 = 92$.

Check.

Running the four blocks of `hw21.examples.py` as printed here and testing the new plotter gives:

- block 1 shows the array above and Figure 1, and cell 9 changes the global paddings to 30/20 only after its first figure; block 2 draws the same size, curves, colours and fonts with paddings 30/20; block 3 saves a figure with two panels;
- `DEFAULT_LAYOUT` has 25 keys; Figure 6 has 4 curves and 2 vertical lines, and the net profits vanish at $S_T = 105$ (call) and $S_T = 92$ (put);
- without layout options: figure size (8,4), solid lines of width 4, font sizes 20/16/16, paddings 30/20 on the figure itself, a dashed line at $y = 0$ and no grid, as in cell 9;
- the unknown key `"colour"` raises `KeyError`; two curves with one name, and `x`, `y` of lengths 3 and 2, raise `ValueError`;
- `np.arange(5)` as both `x` and `y` is drawn as one curve, and `rcParams` is the same before and after a call with a title and an `x`-label;

- two calls with `ax=axes[0]` and `ax=axes[1]` draw into one figure.

Hence the plotter of cell 9 is replaced by

```
my_plotter(x, y, layout=None, names=None, ax=None),
```

which returns `(fig, ax)`. All 25 layout settings of `DEFAULT_LAYOUT` can be customized through `layout`; their defaults are the values of cell 9, so the notebook calls run unchanged and draw the same curves, fonts and sizes (with `tight_layout`). Unknown keys raise `KeyError`, data of the wrong shape raise `ValueError`, integer input works and no global `rcParams` are changed. New features: `y-label`, tick size, colours, line styles, markers, opacity, legend position, grid, reference lines, limits, log axes, saving, subplots.

Theorem 1 (Black–Scholes–Merton price of a European call; Lesson 3, cells 37, 48).

The price at time 0 of a European call with strike K and maturity T on a non-dividend-paying stock is

$$c = S_0 N(d_1) - K e^{-rT} N(d_2), \quad (1)$$

where N is the distribution function of the standard normal distribution and

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2) T}{\sigma \sqrt{T}}, \quad d_2 = d_1 - \sigma \sqrt{T}. \quad (2)$$

Theorem 2 (Greeks of the European call; Lesson 3, cells 46–49).

With N' the density of the standard normal distribution and d_1, d_2 as in (2), the Greeks of the call at time 0 (cell 49 with $t = 0$) are

$$\begin{aligned} \Delta &= N(d_1), & \Gamma &= \frac{N'(d_1)}{S_0 \sigma \sqrt{T}}, & \mathcal{V} &= S_0 N'(d_1) \sqrt{T}, \\ \Theta &= -\frac{S_0 N'(d_1) \sigma}{2\sqrt{T}} - r K e^{-rT} N(d_2), & \rho &= K T e^{-rT} N(d_2). \end{aligned} \quad (3)$$

HW 2/2. Upgrade the BSM call option pricer to be able to return the greeks as well, not just the price.

Solution.

What is asked.

The pricer of cell 43 returns only the price, Theorem 1, (1). It must also be able to return the Greeks of Theorem 2, (3).

A problem in cell 43.

The return line of cell 43 uses `K_vec`, the global variable of cell 44, and not the argument `K`. Cell 44 still works, because there `K=K_vec`, but one strike gives 2000 numbers:

professor's cell 43, run after cell 44

```
# test: one strike K = 20.0 should give one price
c = black_scholes_eur_call(r=0.05, T=1.0, S0=20.0, sigma=0.3, K=20.0)
print(c.shape)
print(c[:3])
```

output

```
(2000,)
[7.66564276 7.66082337 7.65600397]
```

New version.

The code of cell 43 with two changes (the lines of cell 43 that are replaced are kept as `#Delete.` comments):

- **change # 1:** the return line uses the argument `K`, and the price is stored in `price`;
- **change # 2:** a new argument `return_greeks`. With the default `False` the function returns the price, as before, so cell 44 runs unchanged; with `True` it returns a dictionary with the price and the five Greeks of Theorem 2, (3).

new version of cell 43

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
from typing import Dict, List, Union

"""The pricing function of European call option"""
# change # 2
#Delete. def black_scholes_eur_call(r: float, T: float, S0: float, sigma: float,
#Delete.                                     K: Union[float, List[float], np.ndarray]) -> np.ndarray:
# replace by
def black_scholes_eur_call(r: float, T: float, S0: float, sigma: float,
                           K: Union[float, List[float], np.ndarray],
                           return_greeks: bool = False):
# end change # 2
    """
    Black-Scholes pricer of European call option on non-dividend-paying stock

    param r: risk-free interest rate (which is constant)
    param T: time to maturity (in years)
    param S0: initial spot price of the underlying stock
    param sigma: volatility of the underlying stock
    param K: strike price (or prices)
    param return_greeks: if True, return a dict with the price and the greeks
    """
    # check conditions
    assert sigma > 0

    K = np.array([K]) if isinstance(K, float) else np.array(K)

    d1_vec = ( np.log( S0 / K ) + ( r + 0.5 * sigma**2 ) * T ) / ( sigma * T**0.5 )
    d2_vec = d1_vec - sigma * T**0.5

    N_d1_vec = norm.cdf(d1_vec)
    N_d2_vec = norm.cdf(d2_vec)

    # change # 1
    #Delete. return N_d1_vec * S0 - K_vec * np.exp((-1.0)*r*T) * N_d2_vec
    # replace by
    price = N_d1_vec * S0 - K * np.exp((-1.0)*r*T) * N_d2_vec
    # end change # 1

    # change # 2
    if not return_greeks:
        return price
    n_d1_vec = norm.pdf(d1_vec) # N'(d1)
    return {
        "price": price,
        "delta": N_d1_vec,
        "gamma": n_d1_vec / (S0 * sigma * T**0.5),
        "vega": S0 * n_d1_vec * T**0.5,
        "theta": -S0 * n_d1_vec * sigma / (2 * T**0.5) - r * K * np.exp(-r*T) * N_d2_vec,
        "rho": K * T * np.exp(-r*T) * N_d2_vec,
    }
# end change # 2
```

Test.

One strike gives one price, and the running example $S_0 = K = 100$, $r = 0.05$, $\sigma = 0.2$, $T = 1$ gives the price and the Greeks:

test of the new version

```
# test 1: one strike gives one price
print(black_scholes_eur_call(r=0.05, T=1.0, S0=20.0, sigma=0.3, K=20.0))

# test 2: price and greeks, S0 = K = 100, r = 0.05, sigma = 0.2, T = 1
g = black_scholes_eur_call(r=0.05, T=1.0, S0=100.0, sigma=0.2, K=100.0,
                           return_greeks=True)
for name, value in g.items():
    print(name, round(float(value[0]), 6))
```

output

```
[2.84625096]
price 10.450584
delta 0.636831
gamma 0.018762
vega 37.524035
theta -6.414028
rho 53.232482
```

By hand, (2) gives

$$\begin{aligned}d_1 &= \frac{\ln 1 + (0.05 + 0.02) \cdot 1}{0.2} \\&= 0.35, \\d_2 &= 0.35 - 0.2 \\&= 0.15,\end{aligned}$$

and (1) gives

$$\begin{aligned}c &= 100 N(0.35) - 100 e^{-0.05} N(0.15) \\&\approx 63.6831 - 53.2325 \\&= 10.4506,\end{aligned}$$

✓

the price printed above. Cell 44, run unchanged with the new version, draws Figure 7.

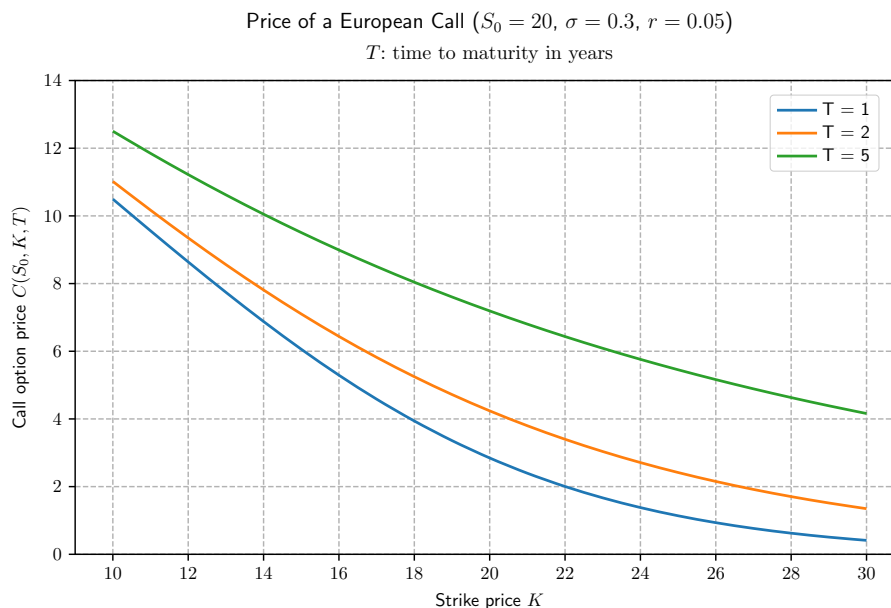


Figure 7: Cell 44 with the new version of the pricer.

The new `black_scholes_eur_call(r, T, S0, sigma, K, return_greeks=False)` returns the price, as cell 43 did, and with `return_greeks=True` a dictionary with the price and the Greeks $\Delta, \Gamma, \mathcal{V}, \Theta, \rho$ of Theorem 2, (3). For $S_0 = K = 100$, $r = 0.05$, $\sigma = 0.2$, $T = 1$:

c	Δ	Γ	$\mathcal{V}$	Θ	ρ
10.450584	0.636831	0.018762	37.524035	-6.414028	53.232482

Theorem 3 (Put–call parity; Lesson 3, cell 30).

For a European call and a European put on the same non-dividend-paying stock, with the same strike K and maturity T , at time 0,

$$c - p = S_0 - Ke^{-rT}. \quad (4)$$

Lemma 1 (Black–Scholes–Merton price of a European put; from Theorems 1 and 3).

With d_1, d_2 as in (2),

$$p = Ke^{-rT}N(-d_2) - S_0N(-d_1). \quad (5)$$

Proof. The standard normal density is symmetric, so $N(-x) = 1 - N(x)$ for every x . We solve (4) for p and insert (1):

$$\begin{aligned}
 p &= c - S_0 + Ke^{-rT} && \text{(Theorem 3, (4))} \\
 &= S_0N(d_1) - Ke^{-rT}N(d_2) - S_0 + Ke^{-rT} && \text{(Theorem 1, (1))} \\
 &= Ke^{-rT}(1 - N(d_2)) - S_0(1 - N(d_1)) && \text{(grouping)} \\
 &= Ke^{-rT}N(-d_2) - S_0N(-d_1). && (1 - N(x) = N(-x)) \quad \square
 \end{aligned}$$

HW 2/3. Implement the BSM pricer for European put option.

Solution.

New version.

The put pricer is the call pricer of cell 43 with three changes:

- change # 1: the name `black_scholes_eur_put` (same arguments);
- change # 2: $N(-d_1)$ and $N(-d_2)$ in place of $N(d_1)$ and $N(d_2)$;
- change # 3: the return line is the put price of Lemma 1, (5) (with the argument K).

the put pricer

```
"""The pricing function of European put option"""
# change # 1
#Delete.    def black_scholes_eur_call(r: float, T: float, S0: float, sigma: float,
#Delete.                                          K: Union[float, List[float], np.ndarray]) -> np.ndarray:
# replace by
def black_scholes_eur_put(r: float, T: float, S0: float, sigma: float,
                          K: Union[float, List[float], np.ndarray]) -> np.ndarray:
# end change # 1
    """
    Black-Scholes pricer of European put option on non-dividend-paying stock

    param r: risk-free interest rate (which is constant)
    param T: time to maturity (in years)
    param S0: initial spot price of the underlying stock
    param sigma: volatility of the underlying stock
    param K: strike price (or prices)
    """
    # check conditions
    assert sigma > 0

    K = np.array([K]) if isinstance(K, float) else np.array(K)

    d1_vec = ( np.log( S0 / K ) + ( r + 0.5 * sigma**2 ) * T ) / ( sigma * T**0.5 )
    d2_vec = d1_vec - sigma * T**0.5

    # change # 2
    #Delete.    N_d1_vec = norm.cdf(d1_vec)
    #Delete.    N_d2_vec = norm.cdf(d2_vec)
    # replace by
    N_minus_d1_vec = norm.cdf(-d1_vec)
    N_minus_d2_vec = norm.cdf(-d2_vec)
    # end change # 2

    # change # 3
    #Delete.    return N_d1_vec * S0 - K_vec * np.exp((-1.0)*r*T) * N_d2_vec
    # replace by
    return K * np.exp((-1.0)*r*T) * N_minus_d2_vec - S0 * N_minus_d1_vec
    # end change # 3
```

Test.

The running example, and the plot of cell 44 for the put (Figure 8):

test of the put pricer

```
# test 1: S0 = K = 100, r = 0.05, sigma = 0.2, T = 1
print(black_scholes_eur_put(r=0.05, T=1.0, S0=100.0, sigma=0.2, K=100.0))

# test 2: the plot of cell 44 for the put
K_vec = np.arange(10, 30, 0.01)
T_vec = [1.0, 2.0, 5.0]

plt.figure(figsize=(10, 6))
for T in T_vec:
    plt.plot(K_vec, black_scholes_eur_put(r=0.05, T=T, S0=20.0, sigma=0.3, K=K_vec),
             label=f"T = {int(T)}")
plt.title('Price of a European Put ($S_0 = 20$, $\sigma = 0.3$, $r = 0.05$)')
plt.xlabel('Strike price $K$')
plt.ylabel('Put option price $P(S_0, K, T)$')
plt.legend()
plt.grid(True)
plt.show()
```

output

[5.57352602]

By hand, with $d_1 = 0.35$, $d_2 = 0.15$ of HW 2/2, (5) gives

$$\begin{aligned} p &= 100 e^{-0.05} N(-0.15) - 100 N(-0.35) \\ &\approx 41.89046 - 36.31694 \\ &\approx 5.5735. \end{aligned}$$

✓

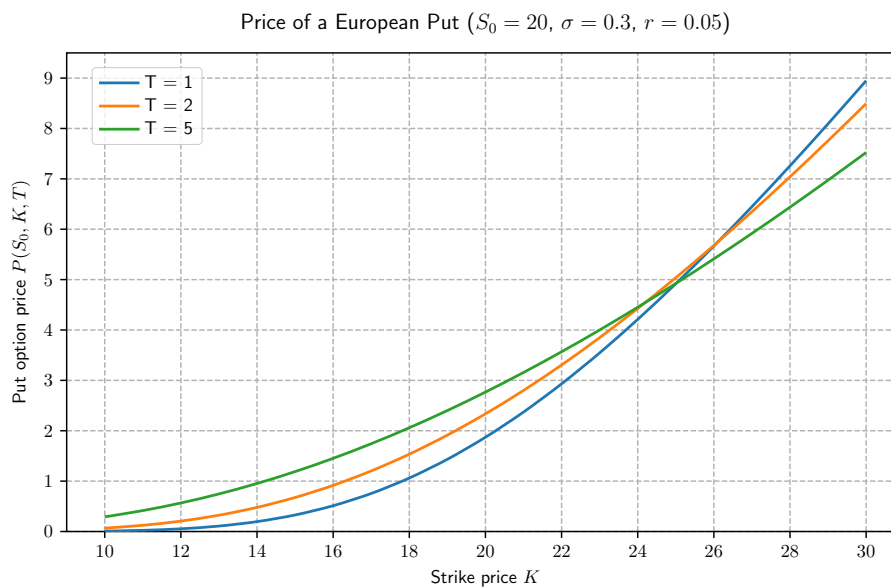


Figure 8: Output of the test: put price against the strike.

The function `black_scholes_eur_put(r, T, S0, sigma, K)` returns the put price of Lemma 1, (5),

$$p = K e^{-rT} N(-d_2) - S_0 N(-d_1).$$

For $S_0 = K = 100$, $r = 0.05$, $\sigma = 0.2$, $T = 1$: $p \approx 5.573526$.

HW 2/4. With the call and put pricer, check if Put-Call parity holds in practice.

Solution.

What is checked.

We compute both sides of Theorem 3, (4): the left side $c - p$ with the two pricers, the right side $S_0 - Ke^{-rT}$ directly, for the strikes and maturities of cell 44 (2000 strikes from 10 to 30, $T = 1, 2, 5$, $S_0 = 20$, $\sigma = 0.3$, $r = 0.05$).

parity check

```
# put-call parity at t = 0: c - p = S0 - K e^{-rT}
K_vec = np.arange(10, 30, 0.01)
T_vec = [1.0, 2.0, 5.0]
S0, sigma, r = 20.0, 0.3, 0.05

plt.figure(figsize=(10, 6))
for T in T_vec:
    c = black_scholes_eur_call(r=r, T=T, S0=S0, sigma=sigma, K=K_vec)
    p = black_scholes_eur_put(r=r, T=T, S0=S0, sigma=sigma, K=K_vec)
    left = c - p # from the pricers
    right = S0 - K_vec * np.exp(-r * T) # from the parity formula
    print(f"T = {int(T)}: max |(c - p) - (S0 - K e^{-rT})| = {np.max(np.abs(left - right)):.1e}")
    plt.plot(K_vec, left, label=f"c - p, T = {int(T)}")
    plt.plot(K_vec, right, 'k--', linewidth=0.8)
plt.title('Put-call parity: $c - p$ (colour) and $S_0 - Ke^{-rT}$ (black dashed)')
plt.xlabel('Strike price $K$')
plt.ylabel('value')
plt.legend()
plt.grid(True)
plt.show()
```

output

```
T = 1: max |(c - p) - (S0 - K e^{-rT})| = 5.3e-15
T = 2: max |(c - p) - (S0 - K e^{-rT})| = 5.3e-15
T = 5: max |(c - p) - (S0 - K e^{-rT})| = 4.4e-15
```

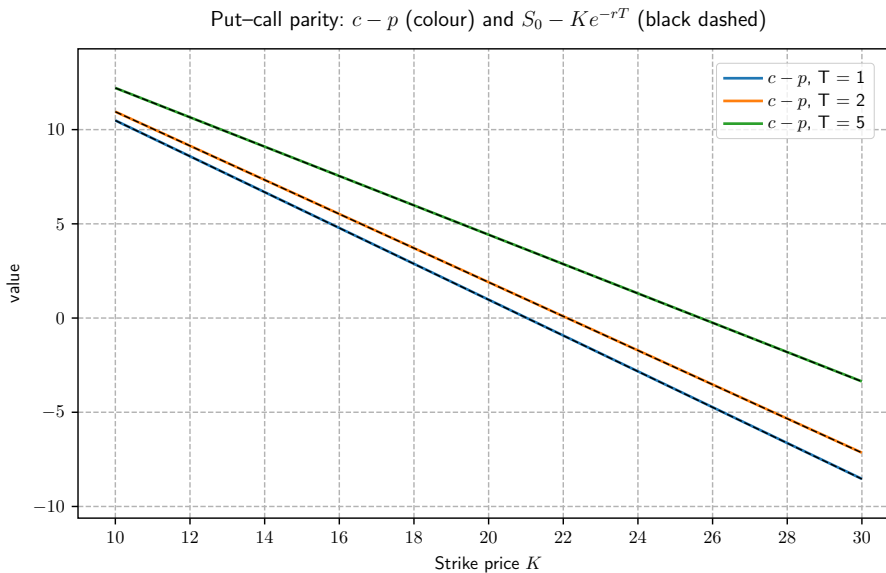


Figure 9: Output of the parity check: the coloured curves $c - p$ from the pricers lie on the dashed lines $S_0 - Ke^{-rT}$.

Why the difference is so small.

By (1), (5) and $N(x) + N(-x) = 1$,

$$\begin{aligned} c - p &= S_0 N(d_1) - Ke^{-rT} N(d_2) - Ke^{-rT} N(-d_2) + S_0 N(-d_1) \\ &= S_0 (N(d_1) + N(-d_1)) - Ke^{-rT} (N(d_2) + N(-d_2)) \\ &= S_0 - Ke^{-rT}. \end{aligned}$$

So the two sides are equal exactly, and the difference of order 10^{-15} printed above is only the rounding error of the computer (about 10^{-16} relative to numbers of size $S_0 = 20$).

Comment. In real markets parity holds only approximately: there are bid–ask spreads and trading costs, many stocks pay dividends and many options are American, and Theorem 3 assumes none of these. For the pricers it holds because the call and the put are priced with the same σ ; pricing them with different volatilities would break it.

Yes. For all 3×2000 contracts of cell 44, our pricers satisfy put–call parity, Theorem 3, (4),

$$c - p = S_0 - Ke^{-rT},$$

up to a difference of at most $5.3 \cdot 10^{-15}$, which is floating-point rounding: for the BSM formulas the two sides are equal exactly.